

Python Assignment-25

1. What is Object-Oriented Programming?

Answer :

Python supports multi-paradigm programming, allowing us to write the code in Procedural ,Object Oriented or Function or Scripting way.

OOP structures code into classes and objects, focusing on real-world modeling using characteristics(attributes) and behavior (methods).

Example:

```
class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def display(self):
        print(self.balance)

acc = BankAccount(1000)
acc.deposit(500)
acc.display()
```

2. What are the four core characteristics of OOP?

Answer :

The four core characteristics (pillars) of Object-Oriented Programming (OOP) are:

1. Encapsulation
2. Abstraction
3. Inheritance
4. Polymorphism

These four concepts are called the Four Pillars of OOP because they form the foundation of object-oriented programming.

3. Explain class and object with real-world analogy.

Answer :

A class is a blueprint (template) for creating objects. It defines the data (attributes) and behavior (methods) that its objects will have.

Suppose Bulider Want to build 100 Houses at Time in same format then he

He make **Blueprint(Plans)** and According ti blue print make staructure plan of constructions And apply it to all.

Blueprint of construction :

House-1

House-2

House-3

...

House-100

4. What is a constructor in Python? Why is `__init__()` used?

Answer :

A constructor is a special method that is automatically called when an object (instance) of a class is created.

"`__init__`" is a reserved method in python classes. It is known as a constructor in object oriented concepts. This method called when an object is created from the class and it allow the class to initialise the attributes of a class.

In python for every instance method there is one implicit parameter as self.

A **Constructor** is a special method that is **automatically called when an object is created**. Its main purpose is to **initialize the object's data (attributes)**.

Syntax :

```
class Student:

    def __init__(self):
        print("Constructor Called")
```

5. Is constructor mandatory in a class? Explain.

Answer :

No. A constructor is **not mandatory** in a Python class. If you do **not** define the `__init__()` method, Python automatically provides a **default initializer** (inherited from the base object class), allowing you to create objects.

6. What is a destructor? When is `__del__()` called?

Answer:

A **Constructor** is a special method that is **automatically called when an object is created**. Its main purpose is to **initialize the object's data (attributes)**.

A **destructor** is a special method that is **called when an object is about to be destroyed (garbage collected)**.

Syntax :

```
class ClassName:

    def __del__(self):
        # Cleanup Code
```

7. Difference between constructor and destructor

Answer :

Constructor	Destructor
<code>__init__()</code>	<code>__del__()</code>
Called during object creation	Called before object destruction
Initializes data	Performs cleanup
Automatically called	Automatically called when appropriate
Runs once per object creation	Runs when object is garbage collected

8. What are instance variables and class variables?

Answer :

The main difference is ownership and memory allocation.

Instance Variables belong to each object.

Class Variables belong to the class and are shared by all objects.

1. Instance Variables (Instance Attributes)

An Instance Variable is a variable that belongs to an individual object (instance).

- Every object has its own copy.
- Changes made to one object's instance variable do not affect other objects.
- They are generally created inside the `__init__()` method using `self`.

2. Class Variables (Class Attributes)

A Class Variable is a variable that belongs to the class itself.

- Only one copy exists.
- It is shared by all objects.
- It is declared directly inside the class, outside any method.

9. Explain **instance methods**, **class methods**, and **static methods**.

Answer :

1. Instance Method

An Instance Method is a method that operates on an object (instance) of a class.

- It can access instance variables.
- It can also access class variables.
- The first parameter must be `self`.

2. Class Method

A Class Method is a method that operates on the class, not on individual objects.

- It can access class variables.
- It cannot directly access instance variables.
- The first parameter is cls.
- It is declared using @classmethod.

3. Static Method

A Static Method is a method that does not operate on either the object or the class.

- It cannot access instance variables using self.
- It cannot access class variables through cls.
- It behaves like a normal function placed inside a class.
- It is declared using @staticmethod.

10. Why do we use @classmethod and @staticmethod decorators?

Answer :

Python provides the **@classmethod** and **@staticmethod** decorators to define methods with different behaviors.

- **@classmethod** creates a method that operates on the **class**.
- **@staticmethod** creates a method that behaves like a **regular function** inside a class and does not operate on either the class or its instances.